

DevZone



Evaluation-driven development (EDD): AI applications that don't lie

CI is green. Your AI is wrong.

EvalHub catches both – live.

William Caban

Sr Principal Product Manager

Red Hat AI



**The unofficial Fennec mascot :)*

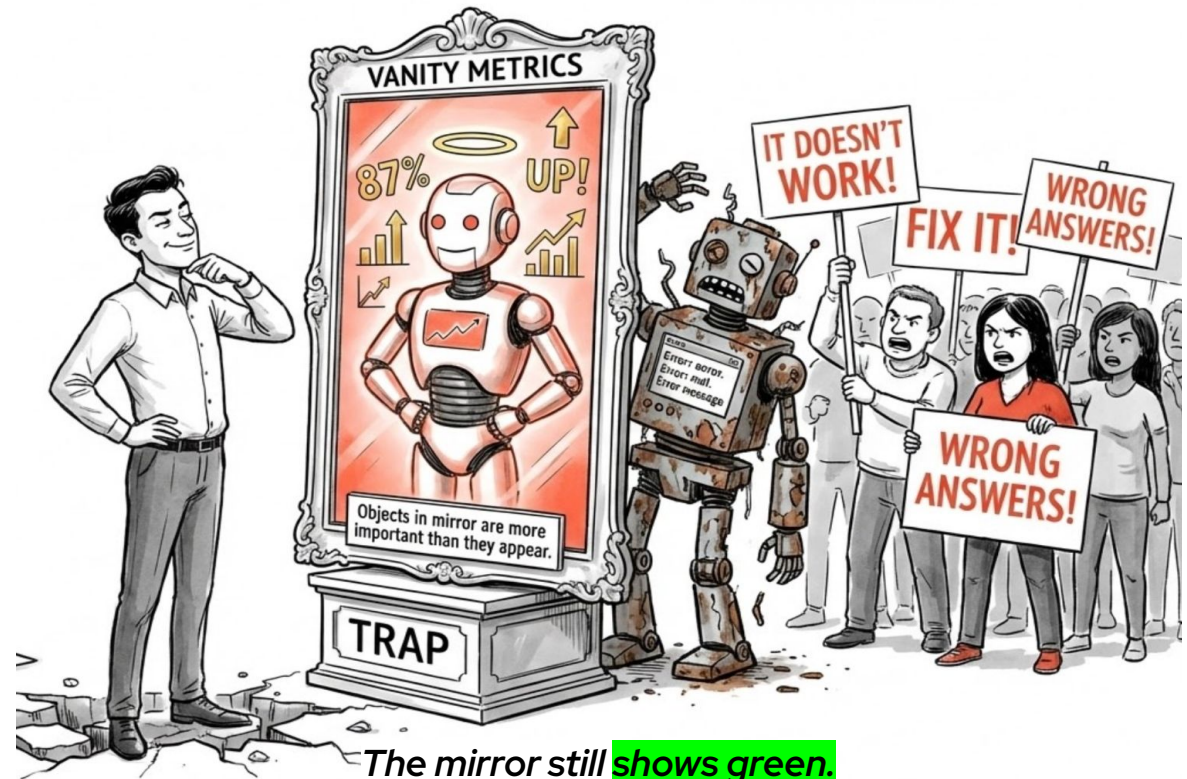
A Familiar Scenario

Your tests pass. Your users find what they don't.

- ✓ Deploy AI assistant.
- ✓ Internal testing passes.
- ✓ Pilot runs smoothly.
- ✓ Stakeholders approve.
- ✗ Three weeks in – user gets the wrong eligibility date.
 - ✗ No error. No alert. CI: still green.

AI in Production: The Hard Truth

- ✗ 60–80% of AI projects never reach production
- ✗ AI projects often fail silently



The mirror still shows green.
But nobody tested whether it was right.

Why Traditional Testing Fails with AI?

Why the Gap Exists? Three Broken Assumptions.

1

Output

TRADITIONAL QA

Deterministic

Standard TDD assumes the same input always produces the same output.

AI REALITY

Non-determinism

Same input. Different outputs. Answer varies with every call.

2

Validation

TRADITIONAL QA

Binary Pass/Fail

Unit tests are absolute. It either works or it doesn't.

AI REALITY

Spectrum Quality

Answers are a metric of how much better or worse.
Spectrum is a range (0-1)
Pass is a threshold of "Good enough"

3

Data

TRADITIONAL QA

Static Fixtures

Hand-written mock data and predictable test environments.

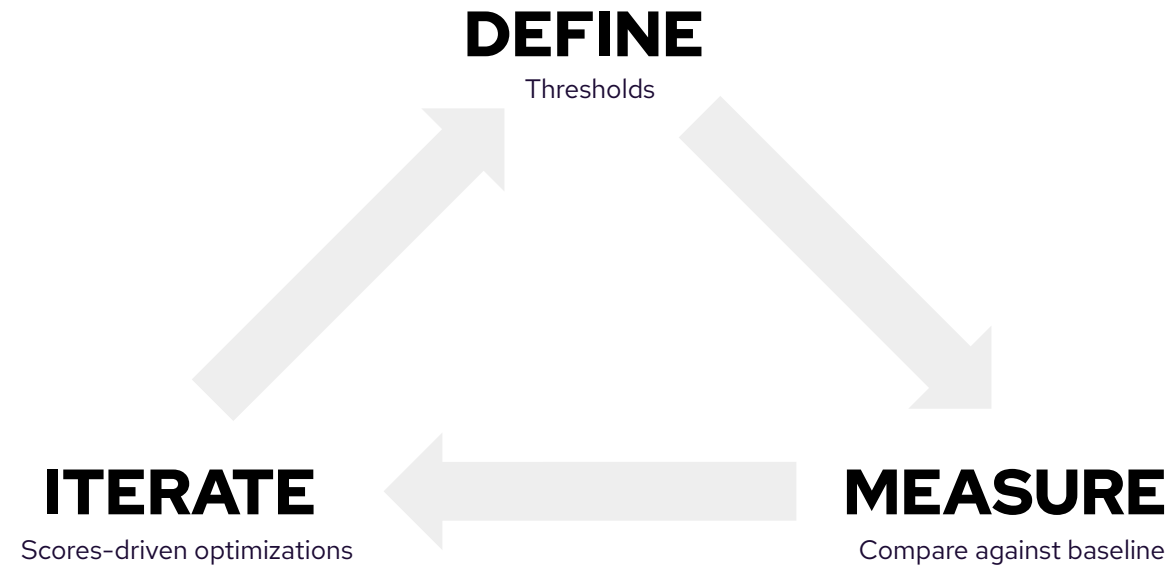
AI REALITY

Dynamic Knowledge

Models don't know what they don't know. Neither does your test suite.
Ground truth must be generated.
Scale makes manual writing impossible.

Introducing Evaluation-Driven Development (EDD)

The EDD Cycle: Define → Measure → Iterate



Every change gets evaluated before it reaches production.
Catches regressions before users do.

EvalHub: Your Precision Evaluation Toolkit

Introduction to EvalHub



**The unofficial Fennec mascot :)*

FRAMEWORK-AGNOSTIC UNIFIED PLATFORM

One place to orchestrate **LM Evaluation Harness**, **LightEval**, **Garak**, **CLEAR**, **GuideLLM**, and custom evaluators via **bring-your-own-framework (BYOF)**

SIMULTANEOUS CAPABILITY + RISK ASSESSMENT

Run capability benchmarks and Garak security/safety probes in the same collection run

BOTTLENECK IDENTIFICATION

Pinpoints exactly where your AI breaks down: model, context, retrieval, or prompts

FULL CONTEXT TRACKING

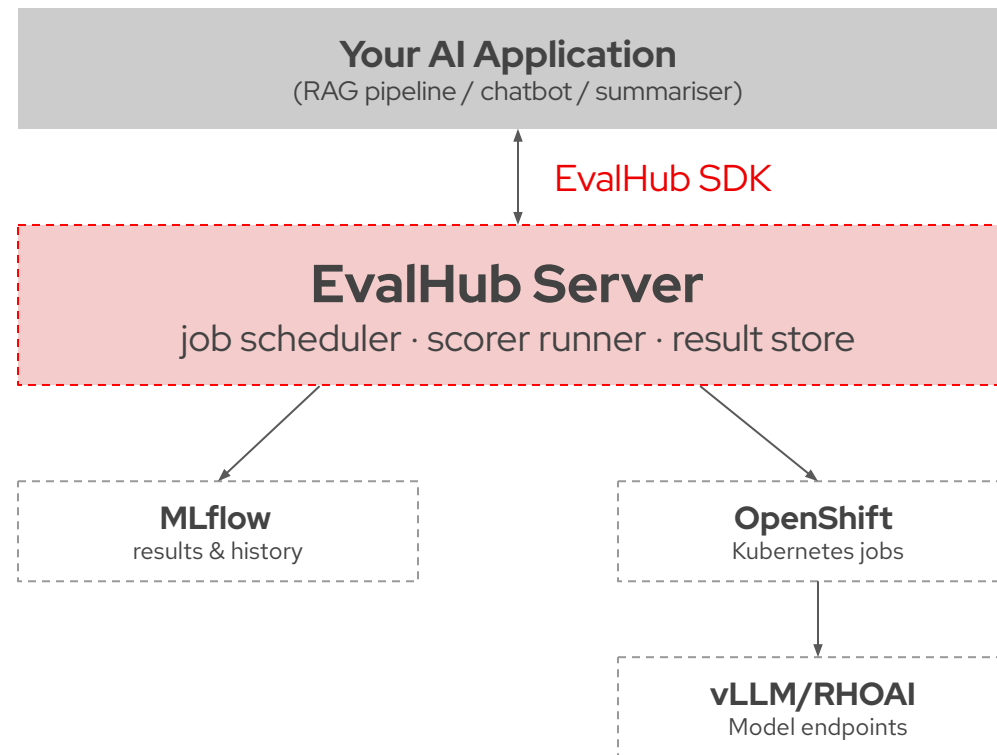
Captures hardware, software, and configuration for every evaluation run via MLflow

EDD-NATIVE

Built for the Define → Measure → Iterate workflow from the ground up

How EvalHub Fits Together

Unified | Scalable | Auditable → From local laptop to OpenShift



The Three Scorers We'll Use

What does EDD actually measure for our demo?



Consistency (Jaccard)

Do consecutive answers agree?
Same question, 5 runs.

Score: 0–1

- 1.0 = identical answers every run
- 0.0 = no words in common between any pair
- Threshold: ≥ 0.50

Catches: non-determinism



Domain F1

Do answers match the ground-truth facts?
Token F1 vs policy.md.

Score: 0–1

- 1.0 = answer perfectly matches ground truth word-for-word
- 0.0 = no overlap with ground truth
- Threshold: ≥ 0.80

Catches: wrong dates and missing info



Confabulation

Does any answer invent facts not in the source?

Score: 0–1

- 1.0 = zero answers contained hallucinated facts (all clean)
- 0.0 = every answer contained a hallucinated date or number
- Computed as: fraction of answers with NO detected confabulation
- Threshold: ≥ 0.90

Catches: hallucinated dates and numbers

Start here. Add domain-specific scorers as you discover.

Act 0 – The Setup

Prerequisites to follow along



```
# Requires: Python 3.12+, uv, podman

$ git clone https://github.com/williamcaban/edd-demo

$ cd edd-demo

$ uv sync

$ podman play kube pod.yaml
```



Act 1 – The TDD illusion

CI passes. The assistant returns the wrong answer – with complete confidence. No test asked if it was right.



```
$ uv run python act1_green_ci.py
```

```
tests/test_pipeline.py::test_returns_a_string PASSED
tests/test_pipeline.py::test_answer_is_not_empty PASSED
tests/test_pipeline.py::test_answer_is_not_a_copy_of_input PASSED
```

```
3 passed in 0.01s ✓ CI would mark this build GREEN
```

Question: "When does open enrollment start and what is the FSA limit?"

```
Call 1: "Open enrollment runs from November 1 through November 15..." ✓ correct answer
Call 2: "Employees can enroll from October 31 to November 15..." ✗ hallucination
Call 3: "Benefits enrollment opens November 1 and closes November 30..." ✗ hallucination
```

CI: still green. Not one test asked if the answer was right.

Act 2 – The reveal

Same question. Different wrong answers. Three scorers find what the tests couldn't.



```
$ uv run python act2_reveal.py
```

```
Consistency Jaccard:      ~0.31  CONCERN  (threshold ≥ 0.50)
```

```
→ answers vary too much run-to-run
```

```
Domain F1 avg:           ~0.42  FAIL     (threshold ≥ 0.80)
```

```
→ wrong dates, missing facts
```

```
Confabulation pass rate: ~0.60  FAIL     (threshold ≥ 0.90)
```

```
→ 2 of 5 answers contain hallucinated dates
```

```
Overall: FAILING – this pipeline should NOT be deployed
```

```
$ uv run python act3_edd_collection.py
```

Act 3 – EDD replaces each broken assumption

Bring your scorer. Register once. EvalHub runs it. Your pipeline gates on the result.



```
$ uv run python act3_edd_collection.py
```

- ✓ Provider registered: hr-rag-byop
- ✓ Collection created: hr-rag-eval-v1 (3 benchmarks, defined once)
- ✓ Job submitted: id=702e5172...

```
BLOCKED - hr-consistency    0.32 < 0.50  
          hr-domain-f1      0.62 < 0.80  
          hr-confabulation  0.60 < 0.90
```

Deploy BLOCKED – all three benchmarks below threshold.

Act 4 – EvalHub at production scale

Scale changes. The verdict doesn't.



```
$ uv run python act4_production.py
```

```
hr-consistency      0.32 FAIL X (1,000 samples)
```

```
hr-domain-f1        0.62 FAIL X (1,000 samples)
```

```
hr-confabulation    0.71 FAIL X (1,000 samples)
```

```
BLOCKED at production gate.
```

Same "evalhub.yaml". Same collection. Same gate. **OpenShift scale.**



Takeaways

1. The AI Gap

Traditional QA is necessary but not sufficient for AI. The gap is structural, not fixable with more test cases.

2. EDD Methodology

Evals-Driven Development gives you a methodology for measuring what matters: faithfulness, relevance, and groundedness.

3. BYOP + Collections

Define your benchmark suite once. Every CI run references it by ID. Thresholds and gate policy travel with the collection, not guessed.

Start Your EDD Journey Today



<https://red.ht/EvalHub>



Thank you

 [linkedin.com/company/red-hat](https://www.linkedin.com/company/red-hat)

 [instagram.com/red_hat](https://www.instagram.com/red_hat)

 [facebook.com/redhat](https://www.facebook.com/redhat)

 x.com/RedHatSummit



red.ht/uxfeedback



Participate in Red Hat AI
UX research opportunities!



EvalHub, because "Looks good to me" Isn't a Benchmark.



EXTRA SLIDES

(backup)

Session ID
LT3001

Title
Evaluation-driven development (EDD): AI applications that don't lie

Abstract

Your tests are green. Your retrieval-augmented generation (RAG) pipeline passed every query in your validation set. Two weeks post-launch: hallucinations, stale answers, retrieval gaps, the demo never surfaced.

The problem is the assumption underlying your tests—that determinism means correctness. Language models don't work that way. Neither do RAG pipelines.

Evaluation-driven development (EDD) is the fix. In this live-coding session, we'll break test-driven development (TDD) against a real large language model (LLM), then rebuild a RAG pipeline with a structured evaluation loop using 2 open source tools:

SDG Hub, a synthetic data generation (SDG) toolkit, which produces domain-specific queries and ground-truth answers at scale

EvalHub, the open source unified AI evaluation platform, which benchmarks faithfulness and retrieval quality, surfaces hallucinations and retrieval gaps, and runs the same evals at production scale on Kubernetes.

Pull out your laptop. We're writing evals from scratch. You'll leave with an eval harness, an SDG Hub flow, and a first look at EvalHub.